ICS 46 · Data Structure Implementation and Analysis · Spring 2026

PA 5,6 - Hash Tables & Heaps - Naming, Tweaking, and Ranking Meals

Units 5 & 6: hash tables and heaps. These files come from AI/statistics problems, but you need none of that background. Everything you need is right here.

This assignment continues the **Menu Planner** (introduced in the one-time *PA + - Overview - The Menu Planner* deck). It spans **two class units**. The Unit 5 half gives every course and dish a **name** (a new `.names` file) and asks you to build **hash maps of those names** that spread them well across the buckets. The Unit 6 half builds an **indexed heap** and puts it to work **ranking**: fast one-dish **tweaks**, the **K best meals** over a meal space far too large to store, and a **settling order** for the courses.

At a Glance

- **Submit on Gradescope: six** files. `NameHash.hpp`, `ModelIndex.hpp`, `IndexedMinHeap.hpp`, `MenuIndex.hpp`, `MealTweaker.hpp`, `BestMeals.hpp`.
- **Given (do not modify):** the menu model (now serving its own course-table index), the `.names` reader `MenuNames`, `tableScore`, `hashSupport`, `IntegerHash`, `HeapEntry`, `mealScan`, and a *complete, working* plain `MinHeap` you read and adapt.
- **Part 1 (Unit 5):** `NameHash.hpp` ships **complete and working**. Your job is to make every name map spread its names well, as described below.
- **Part 2 (Unit 6):** build the `IndexedMinHeap` (adapt `MinHeap`) and `bestMeals`. `ModelIndex.hpp`, `MenuIndex.hpp`, and `MealTweaker.hpp` arrive finished except **one marked line each**.
- **Graded on:** memory safety first (any leak or memory error scores **0**), measured map spread and heap conformance (20), your functions' *return values* against the reference (26), and a small **performance** part (4).

THE NEW .NAMES FILE

Real menus use names, not numbers. Every `.menu` now has a `.names` sidecar next to it (the `.menu` and `.chosen` formats are unchanged):

```
NAMES
<numCourses>
<CourseName> <numDishes> <dishName0> <dishName1> ...
...one line per course, in .menu course order...
```

The sample `samples/menu3.names`:

```
NAMES
3
Appetizer 3 Soup Salad Bread
Main 2 Steak Lentils
Dessert 2 Cake Fruit
```

- Every name is a **single token**. An underscore stands in for a space (for example `Garlic_Bread`).
- Names are **distinct**: unique among the courses, and unique among one course's dishes.
- A name is **2 to 24 characters**, using letters, digits, and underscores.

- Every menu you are given is **valid**: in particular, **every course offers at least one dish**, so every menu can realize at least one complete meal. You never have to handle a course with no dishes.
- No name map is built from more than `MAX_NAMES` (512) names, so its `bucket_count()` never exceeds `MAX_BUCKET_COUNT` (1024). Both constants are in `hashSupport.hpp`; `HASH_PRIME`, far larger than either, is in `IntegerHash.hpp`.
- `numCourses` and each `numDishes` must **match the** `.menu`.

Parsing was PA 0's lesson, so it is **provided**: `MenuNames.hpp` reads the file once and stores every name. After that, call `courseName(c)` for a course's name, `dishName(c, d)` for a dish's name, and `numCourses()` / `numDishes(c)` for the counts. Your work starts after the names are read: you build the fast lookups over them.

WHAT YOU RECEIVE

File	Role
`Course.hpp`, `Meal.hpp`, `MenuModel.hpp`, `mealScore.hpp`, ...	the menu model. New this PA: it **serves its own course-table index**, `tablesWith(c)` **(provided)**
`ModelIndex.hpp`	defines the model's index-builder, `buildCourseTableIndex()` **(one line, submit)**
`MenuNames.hpp`	reads the `.names` sidecar **(provided)**
`tableScore.hpp`	`tableScoreFor(model, t, meal)`, one pairing table's score for a meal **(provided)**
`hashSupport.hpp`	`bucketCountFor` and `courseNeighbors` **(provided)**
`IntegerHash.hpp`	`HASH_PRIME`, `mulmod`, and a complete `IntegerHasher` (the model's index rides it); a worked example worth reading **(provided)**
`HeapEntry.hpp`	one heap entry, a `priority` and a unique `id`, ordered only through `outranks()` **(provided)**
`MinHeap.hpp`	a **complete, working** plain min-heap. Read it and adapt it **(provided)**
`mealScan.hpp`	`startingMeal` and `advanceToNextMeal`: walk the complete meals one at a time **(provided)**
`NameHash.hpp`	**complete and working as shipped (Part 1, submit)**
`IndexedMinHeap.hpp`, `BestMeals.hpp`	skeletons with `// TODO`s **(Part 2, submit)**
`MenuIndex.hpp`, `MealTweaker.hpp`	written for you except **one line each (Part 2, submit)**
`driver.cpp`, `samples/`	local harness and the `menu3` sample set **(provided; do not modify)**

You submit **six** files, but only three carry real building. `NameHash.hpp` is **complete and working** as shipped. `IndexedMinHeap.hpp` and `BestMeals.hpp` are **part-written**: every function is declared and documented, several are already implemented for you, and the ones marked `// TODO` are yours. `ModelIndex.hpp`, `MenuIndex.hpp`, and `MealTweaker.hpp` arrive finished except for **a single line each**, marked `// TODO`, the one decision that matters in each. Among the provided files, `MinHeap.hpp` is a complete, working heap for you to **read and adapt** (the same pattern as PA 3,4's plain list), and `mealScan.hpp` walks the complete meals so you never have to write that loop.

</> PART 1 — THE MAP OF MENU NAMES (NAMEHASH.HPP)

`NameHash.hpp` holds three things. `NameHasher`, the hash class every name map uses. `NameMap<K, V>`, an alias for `std::unordered_map<K, V, NameHasher>`, so every name map carries *your* hasher. And `makeNameMap(keys, values)`, the one function that builds every name map in the assignment. The starter compiles and every lookup works. One requirement remains, and the starter does **not** meet it. It happens to pass on a few of the smallest name lists, and fails on the rest, including every larger one:

What a Good Name Map Looks Like

- **Every lookup stays fast.** A name is found by computing its bucket and looking in that one bucket, so the work per lookup must not grow with the number of names.
- **That needs the names spread out** across the buckets: very few collisions, and no bucket left holding a crowd.
- **Spread is graded in steps.** A map with no pile-ups scores better than one with several. A map where most names sit alone in their own bucket scores best.
- **The autograder tells you where you landed** and what it measured, so you can improve and resubmit.
- `bucket_count()` must **equal** `bucketCountFor(names.size())` and never change afterward.
- **You may change anything you want in this file.**

Rules for `NameHasher` itself:

- It must stay **trivially copyable** and at most **32 bytes** (checked at *compile time* by the autograder).
- It **computes** its answer from the key's **characters** and a few stored **constants**. It may **not** store the names or any per-name data.
- String keys are the exercise; `NameHasher` hashes only them. The one integer-keyed map, the **model's course-table index**, uses the provided `IntegerHasher` entirely.

Mechanically, you edit one small struct. `NameHasher` has a static `hashCode(std::string const&)` returning a `std::uint32_t` (the starter's walks the characters), and an `operator()` the map calls on every key. The struct may carry a few constants as data members (it must stay small and trivially copyable, as above). `makeNameMap` is already written: it presizes the map with `bucketCountFor(n)` buckets and inserts the key/value pairs. You may reshape any of this file as long as those three names keep working.

Grading judges the **map you return**, through the map's own bucket interface: `bucket_count()`, `bucket_size()`, and lookups. It builds maps from several key sets, tiny and large, including sets of *very similar* names, and measures each one. The provided driver prints the very same spread report, so you can watch your spread improve before you submit.

> **Turning a name into a number.** The codes we built in class walk the key one character at a time, keeping a running value: scale what you have so far by a constant, add the next character, and carry on. Scaling before adding is what makes the *order* of the characters matter, so `"TAB"` and `"BAT"` do not land on the same value. Menus are full of names that differ only slightly (a long shared prefix or suffix, one letter apart), so a code that leans on just a few characters, or that ignores their order, gets into trouble here.

PART 2 — INDEXEDMINHEAP, MODELINDEX, MENUINDEX, MEALTWEAKER, BESTMEALS

`IndexedMinHeap.hpp`**: adapt the provided** `MinHeap`**.** `MinHeap.hpp` is a complete, working, array-backed min-heap of `HeapEntry` (level-order array, `_percolateUp`/`_percolateDown`). Read it, then adapt it into an *indexed* heap: alongside the heap array it keeps a **locator** (`id` to that entry's *current slot*), so any entry can be found and re-prioritized quickly by its id.

Mechanically, the skeleton already declares both data members: the heap array `std::vector<HeapEntry> _data`, and the locator `std::unordered_map<long long, int> _locator`. A `HeapEntry` is just a `priority` and an `id`, compared only through `outranks()`. You write the percolation helpers (start from `MinHeap`'s) and the operations below; the read-only accessors are already written. The API:

- `insert(priority, id)`: add the entry. If `id` is **already** in the heap, **change** that id's priority to the given value instead. The entry moves to where it now belongs.
- `size()`, `contains(id)`, `priorityOf(id)` and `peekMin()` are **already written for you**. They have no logic of their own — they simply report what `_data` and `_locator` say — so keep both accurate after every operation and they work by themselves.
- `removeMin()` (throws `std::out_of_range` on an empty heap) and `replaceMin(entry)` (non-empty heaps only) are yours.
- `heapify(data)`: a `static` helper with the same job as `MinHeap::heapify`, turning any array of entries into a heap in place. It is pure array work and knows nothing about the locator.
- `buildFrom(entries)` **replaces the heap's entire contents** with this batch (ids distinct). Whatever was there before is gone, locator included. Afterward the heap holds exactly these entries, every operation above works as usual, and the **locator** describes where every id ended up.

> **Important:** Every ordering decision between two entries goes through `HeapEntry::outranks()`. **Never compare priority fields directly.** (Smaller priority wins. Equal priorities break ties by smaller id.) And every swap your percolates perform must keep the **locator** right. A heap whose array is correct but whose locator is stale fails the conformance tests.

`ModelIndex.hpp`**: the model's own index.** The menu model **serves its own course-table index**: `model.tablesWith(c)` answers "which pairing tables involve course `c`?" in one lookup, the way a real model library keeps such an index inside the model. The code that *builds* it is **yours**: `MenuModel.hpp` only *declares* `buildCourseTableIndex()`; the submitted `ModelIndex.hpp` *defines* it (finished except **one marked line**), and loading a menu calls it. Splitting one member function into its own file is what lets you write this piece of the model without touching the rest of it. (It also means any file that constructs a `MenuModel` must include `ModelIndex.hpp`; the driver and the autograder already do.)

Mechanically: the function shell is given — a loop over the tables (`numTables()`, `factorAt(t)`) and an inner loop over each table's scope (`scopeSize()`, `courseAt(j)`) — with **one marked line**: record table `t` on that course's list in `_tablesByCourse`, the model's `std::unordered_map<int, std::vector<int>, IntegerHasher>` (looking a key up with `[]` creates the course's empty list the first time).

`MenuIndex.hpp`**: the lookups.** Written for you except **one marked line** in `settlingOrder`, and worth reading in full: every map here is a `NameMap` built with *your* `makeNameMap`, and `settlingOrder` drives *your* `IndexedMinHeap`. If either is weak, this code is slow or wrong:

- `NameDirectory`: `courseIndexOf(name)` and `dishIndexOf(courseIdx, name)`, each returning **-1** when the name is unknown.
- `settlingOrder(model)`: repeatedly settle the course with the **fewest unsettled neighbors**, ties to the

lower course index. Two courses are neighbors when they share at least one pairing table. The provided `courseNeighbors(model)` computes the neighbor lists for you, and settling a course lowers each unsettled neighbor's count by one. **Use your** `IndexedMinHeap` to keep the counts.

Mechanically: `NameDirectory` and the loop of `settlingOrder(model)` are written; your **one marked line** sits inside the loop over the settled course's neighbors, where a neighbor's count in the heap is out of date. The heap calls at hand: `contains(id)`, `priorityOf(id)`, and `insert` — remember what `insert` does on an id that is already in the heap.

`MealTweaker.hpp`**: fast one-dish changes, by name.** Written for you except one line, and worth reading closely, because it shows what an index is *for*: it keeps a **per-table score cache**, and `alterDish(courseName, dishName)` returns `false` and changes *nothing* when the course is **locked** (chosen) or when that dish is **already selected**. The names it is given always exist on the menu (requests, like menus, are valid). Otherwise it updates the meal, re-reads **only the swapped course's tables** (the model's `tablesWith(c)` knows which ones), refreshes those cache entries, recombines the total, and returns `true`.

Mechanically, the class is given with its members declared — the name lookups `_dir`, the current `_meal`, the cache `std::vector<double> _tableScore`, the running `_score`, and a finished `_recombine()` that multiplies the cache. Your **one marked line** is the refresh loop inside `alterDish`: which cache entries to re-read with `tableScoreFor(_model, t, _meal)`. The model's `tablesWith(c)` serves a course's tables in one lookup.

> **A pairing score can be 0.** The total is a product of table scores, so think carefully before dividing an old contribution back out of it. Recombining the total from your cache is safe.

`BestMeals.hpp`**: the K best meals.** `bestMeals(model, K, M)` returns the K highest-scoring meals among the **first M** meals the menu can still realize, **best first**. *Can still realize* means the meals that agree with the courses already chosen: a locked course keeps its dish in every one of them. Those meals come in a fixed order, and there can be an **astronomical** number of them, which is why you are asked for the best K out of the first M rather than out of the whole space. `M` counts meals **visited**, so if the menu can realize fewer than M you simply see them all, and you return fewer than K only when fewer than K meals were seen. **The walk itself is provided** (`startingMeal` and `advanceToNextMeal` in `mealScan.hpp`), so what is left to you is the part that matters:

- Score each meal as it goes past, through the **one reusable working** `Meal` the scan hands you, never a fresh `Meal` per candidate. (The provided scan pins every locked course to its chosen dish and steps through the rest in a fixed order, so each realizable meal is visited exactly once.)
- **Never hold more than K** `Meal` **objects** at once.
- Keep the best K with **your** `IndexedMinHeap`, and return them best first.

Mechanically: the loop shell is written — `startingMeal` sets up `working` and `dish`, and `advanceToNextMeal` steps to the next meal — and `mealScore(model, working)` prices the meal going past. What you add inside and after the loop: a small store of the `Meal`s you are keeping (never more than K) and **your** `IndexedMinHeap` of (score, id) entries deciding each replacement.

HOW IT'S GRADED (50 PTS)

Three gates come first. Failing a gate scores **0** for the whole assignment:

1. **Compile gate.** Your six files must compile against the provided headers.
2. **Memory gate.** Everything runs under the address sanitizer. **Any leak or memory error scores 0.**

3. **Source policy.** In your six files you may include *only* `<string>`, `<vector>`, `<unordered_map>`, `<cstdint>`, `<cstddef>`, `<utility>`, `<optional>`, `<stdexcept>`, `<cmath>`, `<fstream>`, plus the project's own headers (the provided ones and your six). **Banned:** the other STL containers (`std::map`, `std::set`, `std::multimap`, `std::multiset`, `std::unordered_set` and the multi variants, `std::list`, `std::forward_list`, `std::deque`, `std::stack`, `std::queue`, `std::priority_queue`, `std::valarray`, `std::bitset`), smart pointers (`std::shared_ptr`, `std::unique_ptr`, `std::weak_ptr`, `std::make_shared`, `std::make_unique`), the library sorting and heap algorithms (`std::sort`, `std::stable_sort`, `std::partial_sort`, `std::nth_element`, `std::qsort`, `std::make_heap`, `std::push_heap`, `std::pop_heap`, `std::sort_heap`, `std::is_heap`, `std::merge`, `std::inplace_merge`), `using namespace`, overloading `operator new` / `operator delete`, `setrlimit`, reading a pairing table directly through `scoreAt` (go through `tableScoreFor`), and **naming any instructor counter** (`scoreCallCount`, `mealCopyCount`, `mealBuildCount`, `tableTouchCount`, `entryCompareCount`).

Then the 50 points:

- **Model tests, 26 pts** (13 tests × 2). The autograder compares your functions' *return values* against the reference on sample and generated menus: name lookups (hits and misses), the model's course-table index, scripted tweak sessions (accepted and refused swaps, plus scores), `bestMeals` for several K (including K larger than the meal count), and settling orders.
- **Unit groups, 20 pts** (10 groups × 2):

Group	pts
Name maps – the map works (bucket count, no rehash, lookups, a plain hasher)	2
Name maps – awkward name sets do not pile up	2
Name maps – no crowded buckets anywhere	2
Name maps – most names sit alone	2
IndexedMinHeap – insert / peekMin / removeMin ordering	2
IndexedMinHeap – equal priorities break ties by id	2
IndexedMinHeap – insert on an existing id changes its priority	2
IndexedMinHeap – locator stays right under mixed operations	2
IndexedMinHeap – buildFrom replaces the whole heap	2
Meals – ties, a zero score, locked courses, and edge sizes	2

- **Performance, 4 pts.** A small part of the grade rewards an efficient organization of the work.

> **Important:** Know your own code. You may be **asked about your implementation** in class: how your hash spreads the names, what the locator buys your heap and what every swap owes it, and why your tweaks and your ranking do so little work. Be ready to explain each, and *why*.

SUBMIT

What to Turn In

Submit on **Gradescope** (the **PA 5,6** assignment). Upload **exactly these six files**: `NameHash.hpp`, `ModelIndex.hpp`, `IndexedMinHeap.hpp`, `MenuIndex.hpp`, `MealTweaker.hpp`, and `BestMeals.hpp`. The filenames must match **exactly**. You may resubmit as often as you like before the deadline; the

autograder runs on each submission and shows you the full per-test results.

>_ BUILD & TEST LOCALLY

```
cd "PA 5,6 - Hash Tables & Heaps - Naming, Tweaking, and Ranking Meals/starter"
g++ -std=c++20 -Wall -Wextra -o driver driver.cpp
./driver samples/menu3
```

The provided `driver.cpp` loads the sample menu, its names, and its chosen courses, then exercises every piece you build: it prints each name map's **spread** through the same bucket interface the autograder reads, the name lookups (hits and misses), each course's table list, a scripted tweak session with a full-rescore self-check, the best meals, the settling order, and the work counters. Run it as you work to watch each part come alive, and **compare your run against the provided** `expected_output.txt` — the driver's full output for a finished solution. Its header marks the few lines that legitimately differ (the spread lines depend on your hash; the work counters on your design); everything else should match exactly. Check for leaks with `-fsanitize=address,undefined`.

> **Grading environment.** Your code is compiled and run on Gradescope in **Ubuntu 22.04** with `g++` **11** at `-std=c++20`. Make sure all six files compile cleanly there.